

TEAM

Tech Mavericks

AegisLab

Controlled, measurable, adversarial AI testing infrastructure

The challenge: red-team attacks, blue-team defences and the model under test must coexist without leaking protected state, or the evaluation itself becomes invalid. AegisLab keeps them apart, measures what crosses, and hands the reviewer evidence that can be checked.

Hack in Hills ’26 · Track 04
Securing Adversarial AI Safety Testing Infrastructure (Bayora)

AM

Abdul Munaf Z

Solution architect & integration engineer
Threat model, trusted broker, API integration, overall demonstration

LS

Likhith Sai N

Sandbox security & infrastructure engineer
Workload isolation, network policies, identities, resource controls, deployment

SR

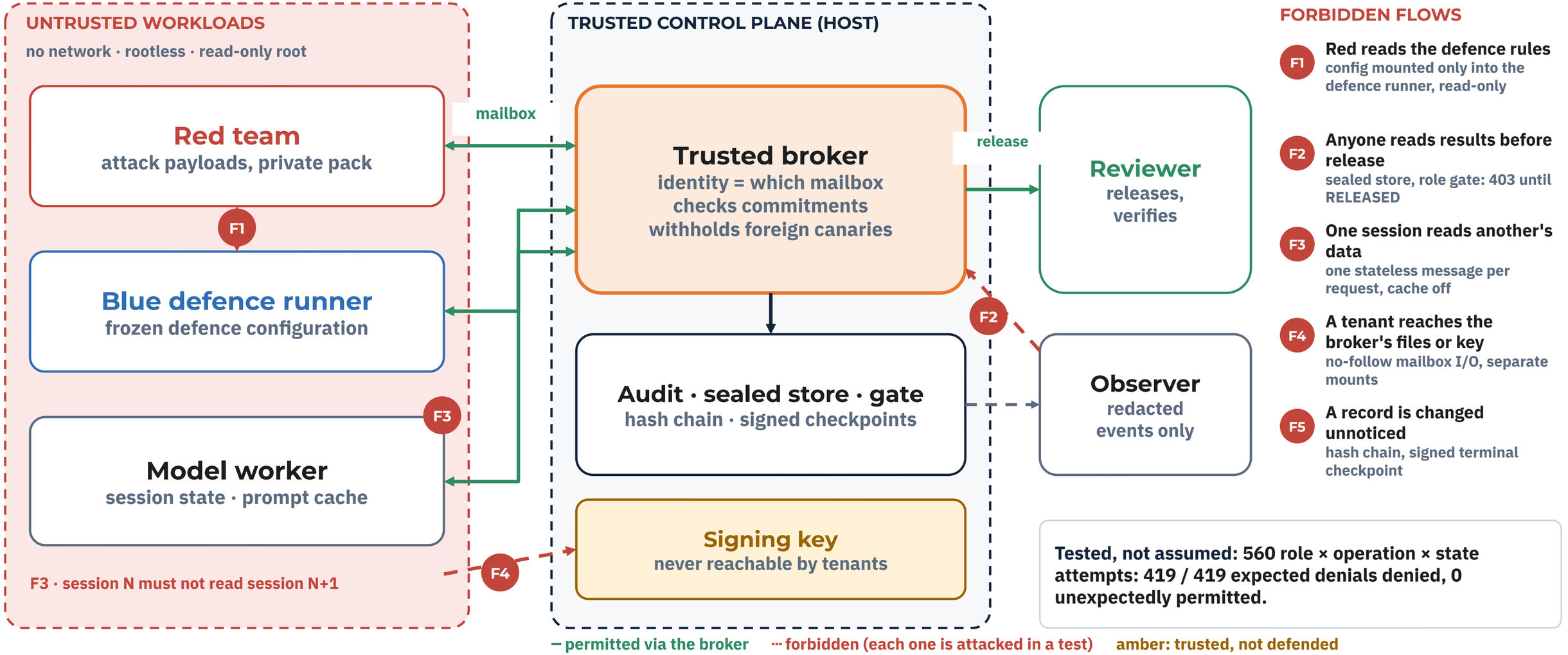
Sreeram Rajesh

Product interface & evaluation engineer
Website, run reports, audit verification, test harness, presentation assets

Roles are the responsibilities each member owns in this project.

THREAT MODEL

Who must not see what, and where each rule is enforced



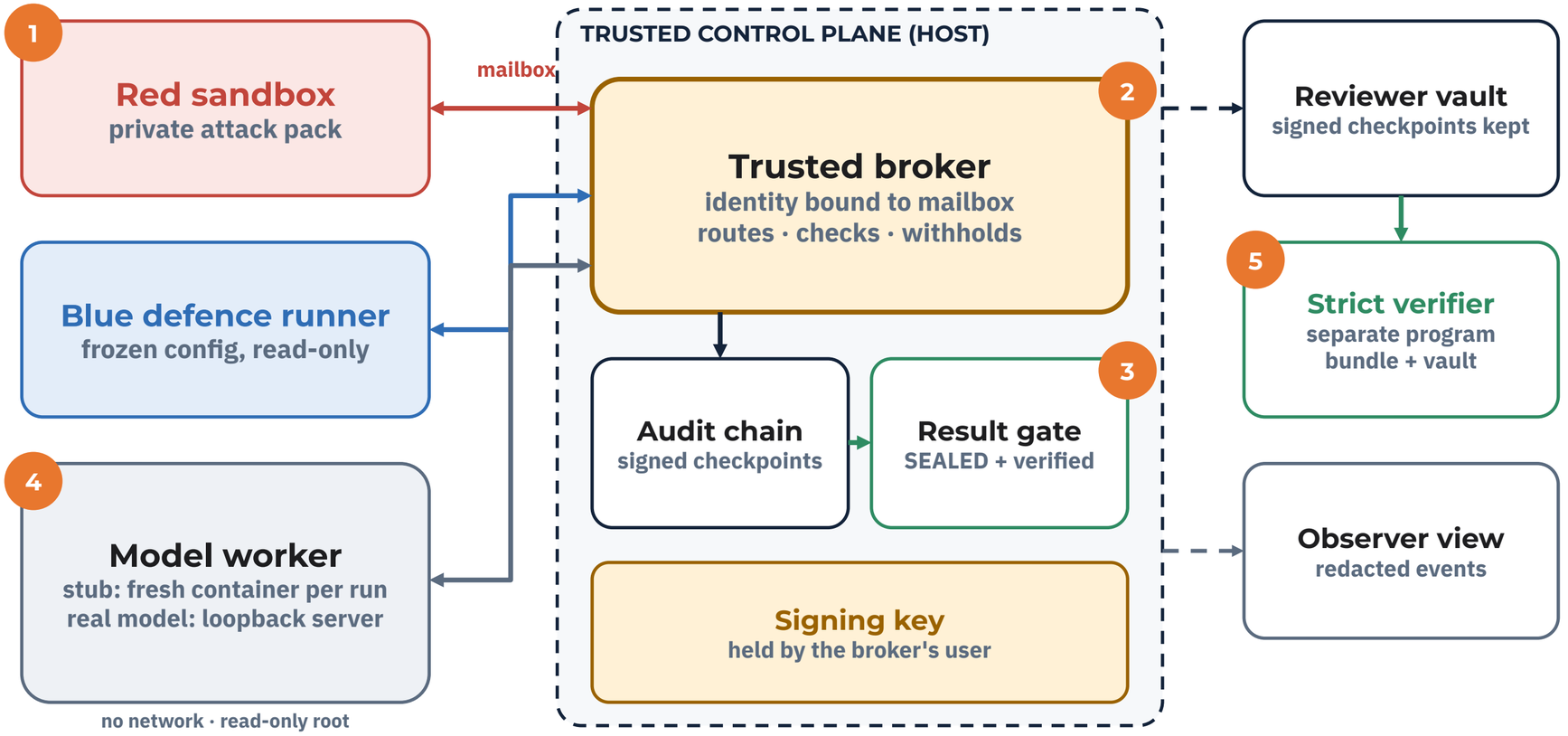
FORBIDDEN FLOWS

- **F1** Red reads the defence rules
  config mounted only into the defence runner, read-only
- **F2** Anyone reads results before release
  sealed store, role gate: 403 until RELEASED
- **F3** One session reads another's data
  one stateless message per request, cache off
- **F4** A tenant reaches the broker's files or key
  no-follow mailbox I/O, separate mounts
- **F5** A record is changed unnoticed
  hash chain, signed terminal checkpoint

Tested, not assumed: 560 role × operation × state attempts: 419 / 419 expected denials denied, 0 unexpectedly permitted.

ARCHITECTURE

Five enforcement layers around one trusted broker



RESIDUAL TRUST (not defended) shared host kernel · container runtime · the trusted broker and its signing key

WHAT EACH LAYER ENFORCES

1. **Container limits**
   rootless Podman, --network none, read-only root, no capabilities, no-new-privileges, pids/memory/CPU caps
2. **Controlled broker path**
   file mailbox; in/ and out/ separate mounts; no-follow descriptor I/O; size and type checks
3. **Role and run identity**
   identity from the mailbox, not the message; server-side role gate; release only when SEALED and verified
4. **Model and session isolation**
   one user message per request; no session id; cache_prompt false; foreign canaries withheld
5. **Audit and verification**
   hash chain, signed terminal checkpoint, retained copies, independent strict verifier

ADVERSARIAL DEMONSTRATION

A hostile sandbox plants a symlink and a FIFO in the broker's mailbox

THREAT — Tenant code turns the broker's reply write into an overwrite of the audit log, or hangs the broker.

PROTECTED — audit.jsonl (the hash-chained run log) and the broker's availability.

ATTEMPT — Real rootless container, no network, read-only root: plant a symlink and a FIFO at guessed temp names; rename or remove the mailbox directory.

RESULT — Planted entries ignored; replies delivered as regular files; victim file intact; nothing created outside; the broker never blocked.

TESTS — P2B01 · P2B03 · P2B05 · P2B32 · P2C01 · P2C02, all in the suite (345 passed).

HISTORICAL · before the fix

The same attack on the Phase 1 broker made it overwrite the audit log: victim audit.jsonl intact: False (recorded 23 Oct, evidence/phase2_2026-10-23/before_fix/11_poc_mailbox.txt). Found by our own Phase 2 attack, then fixed.

scripts/judge_demo.sh · step 3 · hostile mailbox

```
hostile mailbox: a real rootless sandbox (no network, read-
    only root, no capabilities) attacks the broker's mailbox
  inside sandbox: symlink_at_guessed_temp_name   succeeded
  inside sandbox: fifo_at_guessed_temp_name      succeeded
  inside sandbox: rename_mailbox_dir             refused
      (errno 30)
  inside sandbox: remove_mailbox_dir             refused
      (errno 30)
[PASS] broker replies were delivered as regular files
    despite the planted symlink and FIFO: ['000002.json',
    '000003.json']
[PASS] the victim file outside the mailbox is intact
    (symlink not followed)
[PASS] nothing was created outside the mailbox: []
[PASS] the broker did not block on the FIFO: 0.000 s for two
     writes
```

RECORDED live run of scripts/judge_demo.sh, step 3 · stub model · current code · 23 Oct 2026

Also blocked: 282 / 282 replayed model-generated release messages were denied by the gate (real-model run, E4).

REAL-MODEL EVIDENCE

Real model: no cross-session leak seen, and the controls show it could

REAL MODEL · NOT THE STUB Qwen2.5-0.5B Q4_K_M · llama.cpp on CPU · loopback only

A · DIRECT ADAPTER (16 sessions per cell, 10 probe wordings per session)

Isolation: 7 cells, fresh and reused server, cache on and off

0 / 104

sessions that recovered another session's canary · 1,040 trials

Negative control: nothing planted

0 / 32

320 trials

Positive control: history shared on purpose MUST LEAK

16 / 16

106 of 160 trials leaked

Positive control: context-sharing gateway MUST LEAK

13 / 16

60 of 160 trials leaked; the first session cannot leak by construction

B · FULL PIPELINE (broker + rootless sandboxes + real model, 12 attacks per run)

Isolation runs
0 / 26 runs
foreign canary in raw model output · 312 attacks

Negative control
0 / 4 runs
foreign canary in raw model output · nothing planted

Gateway control
2 / 4 runs
foreign canary in raw model output · 15 withheld, 0 reached red

READ THIS WITH THE LIMITS

Not proof of zero. One-sided 95% upper bound: 2.84% of sessions (0.29% of trials), for this probe set, model and server.

Weak probes. Q01, Q08, Q10 (3 of 10) stayed silent even when context was shared on purpose; their zeros carry no weight.

What it tests. Application-level session isolation in this setup. Not GPU memory, accelerator KV-cache or hardware side channels: untested.

Injection. 72 attacks, 0 protected values in red-visible output or monitoring: an information-flow result, not model robustness.

Failure handling. 9 / 9 model-outage checks hold.

AUDIT INTEGRITY & REPRODUCIBILITY

A released run verifies; with 3 entries removed it does not

COMPLETE RUN · exit 0

verify_cli · complete run

```
$ python -m aegis.verify_cli --bundle <bundle>
    --run-id JD-1 --expect-state RELEASED --pubkey
     <pinned> --expect-terminal-hash <noted at
    release>
  ok                   True
  entries              28
  checkpoints_checked  6
  terminal_verified    True
  terminal_state       RELEASED
  key_source           pinned (--pubkey)
[PASS] strict verification of the complete bundle
    (key and terminal hash pinned): exit 0
```

TAMPERED COPY · exit 1

verify_cli · tampered copy

```
$ python -m aegis.verify_cli --bundle <tampered
    copy> ... (same options)
  ok                   False
  entries              25
  checkpoints_checked  6
  terminal_verified    False
  terminal_state       RELEASED
  key_source           pinned (--pubkey)
  finding: log has 25 entries but a signed
      checkpoint covers seq 26: truncation
  finding: log has 25 entries but a signed
      checkpoint covers seq 27: truncation
  finding: log has 25 entries but a signed
      checkpoint covers seq 28: truncation
[PASS] verification of the tampered copy FAILED,
    as it must: exit 1
[PASS] the original bundle still verifies
```

RECORDED scripts/judge_demo.sh, steps 4 and 5 · run JD-1 · key and terminal hash pinned by the reviewer · 3 entries removed from a copy

run RELEASED → terminal checkpoint signed → bundle exported → strict verify: pinned key + noted hash

REPRODUCE IT

bash scripts/quickstart.sh
8 / 8 stages pass; 345 tests passed

bash scripts/judge_demo.sh
17 / 17 checks, a few seconds, offline

bash scripts/security_demo.sh
34 / 34 checks plus 1 documented limit

python evaluation/check_claims.py
recomputes quoted figures from raw files (134 / 134 checks)

In progress is not complete. A log still being written is reported terminal_verified: false and never counts as a strict pass; the legacy DEMO-001 predates terminal checkpoints and is rejected by design.

PERFORMANCE, LIMITS & DEPLOYMENT

What it costs, and exactly where the guarantees stop

MEASURED COST (one host, CPU)

Broker hop
2.59–2.84 ms median, idle host, stub model

Timing padding, 250 ms
Classifier accuracy 0.495–0.555 vs 0.795–0.800 unpadded; +198 to +201 ms per answered request. Lowers one classifier's accuracy; does not show the timing channel is gone.

Real model on CPU
Qwen2.5-0.5B on a loopback-only endpoint: median 349 ms per request, prompt cache off.

Inference state
cache_prompt true: a second session confirms a guess (cached tokens 119 vs 99, AUC 1.00). AegisLab sends false: 0 / 120 trials cached, 0 / 14 after restart.

RESIDUAL RISK: NOT PROVEN AWAY

Trusted broker and signing key
A compromised broker signs false but consistent records. A key thief who replaces every copy defeats the scheme unless the reviewer holds an independent terminal hash.

Shared host
Kernel, container runtime and hardware side channels are shared. GPU memory and accelerator KV cache were not tested.

One model, one detector
One 0.5B model and ten probe wordings; 3 probes silent under shared context. The canary detector misses re-encodings such as base64.

DEPLOYMENT

TODAY · implemented
One host, rootless Podman, API on loopback only, JSON files, local demo tokens. Real model optional, loopback only, key never logged.

NEXT · proposed, not built
Signer outside the control-plane user · run-scoped identities · vault on separate storage · refuse cache_prompt true unless explicitly shared · microVM per tenant · GPU model server behind its own boundary



AEGISLAB · TECH MAVERICKS

Three contributions, with their limits stated

1. **Enforced information-flow boundaries**
   No-network sandboxes, one brokered path. 419 / 419 expected denials denied in a 560-case role matrix; the hostile-mailbox attack is blocked in a real sandbox.

2. **Reproducible adversarial validation, real model included**
   0 / 104 sessions leaked, and the controls that must leak did (16 / 16, 13 / 16). One command reproduces it.

3. **Verifiable evidence with explicit residual risk**
   A signed terminal checkpoint; a tampered copy is rejected. Trusted broker, shared host, detector limits and untested hardware are stated.

Why it matters: a safety finding is only as credible as the test that produced it.

Abdul Munaf Z · Likhith Sai N · Sreeram Rajesh

Thank you!